

EdgeMM: Multi-Core CPU with Heterogeneous AI-Extension and Activation-aware Weight Pruning for Multimodal LLMs at Edge

Kangbo Bai

School of Integrated Circuits
Peking University
Beijing, China
baikangbo@pku.edu.cn

Le Ye

School of Integrated Circuits
Peking University
Beijing, China
yle@pku.edu.cn

Ru Huang

School of Integrated Circuits
Peking University
Beijing, China
ruhuang@pku.edu.cn

Tianyu Jia*

School of Integrated Circuits
Peking University
Beijing, China
tianyuj@pku.edu.cn

Abstract—Emerging multimodal LLMs (MLLMs) exhibit strong cross-modality perception and reasoning capabilities and hold great potential for various applications at edge. However, MLLMs typically consist of a compute-intensive modality encoder and a memory-bound LLM decoder, leading to distinct bottlenecks for hardware designs. In this work, we present a multi-core CPU solution with heterogeneous AI extensions, which are based on either the compute-centric systolic array or memory-centric digital compute-in-memory (CIM) coprocessors. In addition, dynamic activation-aware weight pruning and bandwidth management are developed to enhance bandwidth efficiency and core utilization, improving overall performance. We implemented our solution using commercial 22nm technology. For representative MLLMs, our evaluations show EdgeMM can achieve $2.84\times$ performance speedup compared to laptop 3060 GPU.

Index Terms—Multimodal Large Language Model, Multi-Core CPU, Heterogeneous Architecture, CPU Extension.

I. INTRODUCTION

Multimodal large language models (MLLMs) exhibit strong perception, reasoning, and planning abilities, with promising applications in autonomous driving [1], embodied AI [2], mobile [3], and AR/VR. As shown in Fig. 1 (a), MLLMs take in various modalities of input, such as image, audio, and LiDAR, which are processed by modality encoder to generate token representations. Following LLM and post-processing are performed for tasks such as comprehension, object segmentation [4], or trajectory planning [5]. Due to the application demands for privacy and real-time responses, deploying MLLMs on edge devices is highly desired.

Achieving low-latency inference of MLLMs at edge is challenging. First, the hardware resource is quite constrained at edge. Second, different phases in MLLMs exhibit opposite characteristics and performance bottlenecks. For instance, the encoder network is compute-intensive, while the LLM-decoding phase is bounded by the memory bandwidth. Such workload behaviors limit performance improvements of certain accelerators tailored for particular Transformers [6], [7]. Meanwhile, the prior optimization techniques for long-context

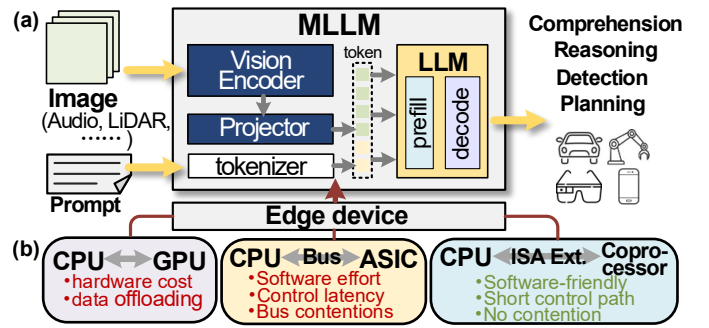


Fig. 1. MLLMs and hardware architecture design choices.

LLMs are not suitable for edge MLLMs, which process short token sequences with streaming inputs.

There are multiple hardware architecture choices for edge devices, as shown in Fig. 1(b). CPU remains the most common hardware in edge devices, while it has limited parallelism for AI tasks. The use of GPU on edge devices is severely limited by cost, and data offloading from CPU to GPU may become a system bottleneck [8]. ASIC accelerator is also a popular solution [9], which is connected to host CPU through memory-mapped interface. However, ASIC accelerators require corresponding compiler development, adding huge engineering effort for software developers. They also introduce bus access with increased control latency and bus contention [10].

In recent years, AI-target ISA extension with in-CPU coprocessors has emerged as a promising solution for AI tasks. For example, Intel introduced its advanced matrix extensions (AMX) [11] in the Xeon processors [12], [13]. Arm developed scalable matrix extension (SME) in Armv9-A [14]. T-head proposed open-source RISC-V matrix extension [15]. At architecture level, the host CPU cores decode and dispatch these extended instructions to coprocessors for execution via direct-linked interfaces. Compared to conventional bus-attached accelerators, coprocessors have a shorter control path and avoid bus contention. Moreover, this architecture can be merged into existing CPU ISA and inherit the compiler stack [13].

In this work, we explore a multi-core CPU solution for

*Corresponding author.

TABLE I
REPRESENTATIVE MLLMS AND EFFICIENT EDGE MLLMS.

MLLMs	Visual Encoder	Projector	Language Model
Emu2-Chat [16]	EVA [17]	MLP	LLaMa (33B)
LLaVA [18]	CLIP ViT/L-14 [19] (0.3B)	MLP	Vicuna [20] (7B, 13B)
MobileVLM [21]	CLIP ViT/L-14 [19] (0.3B)	LDP	MobileLLaMA [21] (2.7B)
TinyGPT-V [22]	EVA	Q-former [23]	Phi-2 [24] (2.7B)
SPHINX-Tiny [25]	CLIP ViT/L-14 [19], CLIP ConvNeXt [19]	MLP	TinyLlama [27] (1.1B)
DeepSeek-VL [28]	+ DINOv2(0.4B) [26]	MLP	DeepSeek-LLM [28] (1.3B)
Karmavlm [30]	SigLIP-L [29] (0.4B), SigLIP-so [29] (0.4B), CLIP ViT/L-14 [19] (0.3B)	MLP	Qwen1.5 [31] (0.5B)

edge MLLMs with heterogeneous AI-extension. For compute-intensive general matrix multiplication (GEMM), systolic array is utilized as the coprocessor. Alongside, for memory-bound matrix-vector multiplication (GEMV), which dominates LLM-decoding phase, area- and power-efficient digital compute-in-memory (CIM) macros are adopted. These coprocessors have different dataflows and resource distributions, coupled with RISC-V ISA and programming model for easy deployment. Furthermore, a dynamic activation-aware weight pruning is developed along with hardware support to alleviate memory bandwidth pressure. At the system level, the DRAM bandwidth can be dynamically managed to fully utilize heterogeneous cores for diverse sequence lengths.

The contributions of this paper are summarized as follows:

- We analyze the attributes of edge MLLMs, revealing that different phases encounter distinct bottlenecks in computation and memory access.
- We develop the heterogeneous extension for multi-core CPU, employing the systolic array and CIM macro tailored for GEMM and GEMV.
- We present an activation-aware weight pruning and bandwidth management, leveraging the limited bandwidth and heterogeneous cores to accelerate edge MLLM.
- EdgeMM is implemented and evaluated at 22nm, achieving a 0.217 token/J efficiency and 138 tokens/s throughput, which is $2.84\times$ better than RTX 3060 GPU.

II. BACKGROUND AND MOTIVATIONS

A. Multimodal Large Language Models

MLLM builds on standard LLM by incorporating multimodal data for input and output. In recent two years, leading MLLMs such as Gemini [32], GPT-V [33], LLaVA [18], InstructBLIP [34] have emerged, impacting both industry and academia. Combining LLMs' reasoning with multimodal perception, they exhibit remarkable performance.

Fig. 1(a) illustrates the typical backbone architecture of MLLM, which is composed of modality encoder, projector, and LLM. Take visual modality as an example. The features are extracted from input images by Transformer-based vision encoder, e.g. CLIP [19], DINOv2 [26] and then aligned with language model via a projector, e.g. MLP, Q-former [23], to generate vision tokens. The vision tokens and text tokens from the prompt are jointly fed into the LLM. The inference of LLM consists of two phases, i.e. prefill and decoding. In the prefill phase, attention is applied between all tokens, generating the first output token and key-value (KV) cache. In the decoding phase, only the last token is taken as input, autoregressively generating the next token and updating the KV cache. The

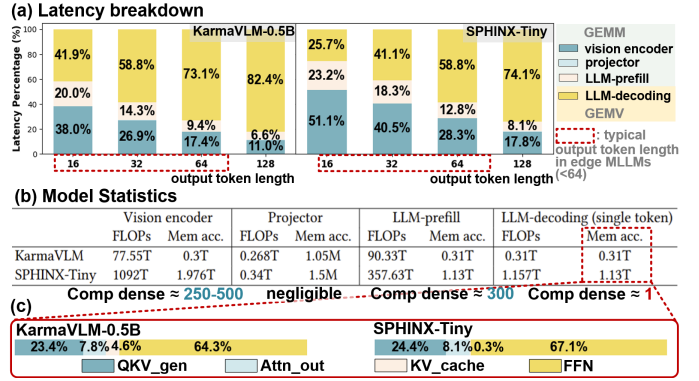


Fig. 2. Workload analysis of two MLLMs: (a) latency breakdown, (b) model statistics, (c) memory accesses.

generated tokens can be converted into language as output or further processed by downstream tasks, e.g. AD [1].

Tab. I lists a few representative open-source MLLMs and their model architectures. MLLMs [16], [18] often adopt LLMs with parameter amounts surpassing 7B. Meanwhile, there is also a model development trend for lightweight MLLMs to assist the deployment at edge. These edge MLLMs adopt compressed LLMs with less than 3B parameters, such as Phi2-2.7B [24], TinyLlama-1.1B [27], Qwen1.5-0.5B [31]. It is also worth pointing out that these edge MLLMs also demonstrate comparable performance compared to large-scale MLLMs in many benchmarks [35]–[38].

B. Model Profiling and Analysis

To better understand the workload characteristics, we conduct profiling on two representative MLLMs: SPHINX-Tiny [25] and Karmavlm [30]. The input token length is about 300, primarily made up of vision tokens. The inference latency breakdown on Nvidia 3060 GPU with varying output token lengths is shown in Fig. 2(a). An increase in output tokens leads to more LLM-decoding iterations, enlarging the proportion of this phase in overall latency. Alongside, the vision encoder and LLM-prefill also contribute to the overall latency notably in edge applications with short token lengths. The latency of the projector is negligible.

Fig. 2(b) also indicates the computing and memory attributes differ significantly during different phases. In vision encoder and LLM-prefill, multiple token embedding vectors share weight matrices forming computation-intensive GEMM. Meanwhile, LLM-decoding performs the operations between the entire weight matrix and a single embedding vector, resulting in low-computational-density GEMV. The LLM-decoding employs the same model parameters as LLM-prefill but involves two orders of magnitude fewer FLOPs, revealing an obvious memory-bound bottleneck. As shown in Fig. 2(c), weight matrices dominate the memory access, with FFN consuming the largest portion as the channel dimension is typically several times larger than the model dimension [39]. The KV cache only contributes a small portion of memory access due to the short token length in edge MLLMs.

Since the majority of DRAM access is occupied by FFN weight matrices, we further analyze the characteristics of this

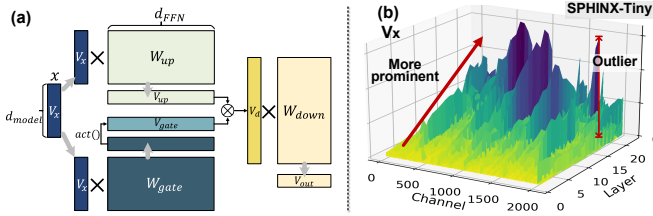


Fig. 3. Gated-MLP and the activation vector sparsity in FFN.

portion, e.g. sparsity. Currently the mainstream LLMs, e.g. Llama series [40], Mistral [41], Qwen [31], use the gated-MLP as FFN. Fig 3 (a) shows the detailed process, which contains three GEMV, and the calculation formula is as follows:

$$FFN(V_x) = ((V_x * W_{up}) \circ (act(V_x * W_{gate}))) * W_{down} \quad (1)$$

where V_x is the input vector, W_{up}, W_{gate} are $d_{FFN} \times d_{model}$ weight matrices. $act()$ is the activation function. In fact, the activation vectors (V_x, V_d) in GEMV exhibit notable sparsity across channels. Fig. 3(b) shows profiled magnitudes of V_x during a token generation in SPHINX-Tiny [25]. The x, y, and z axes denote the decoder layer index, vector channels, and absolute magnitudes. As presented, these activation vectors exhibit notable sparsity for most channels, with a few outlier channels that can be easily masked out. Moreover, as the layer index increases, these outliers become more prominent.

The above analysis guides us to rethink the optimal architecture for edge MLLMs with the following design insights.

Insight 1: Heterogeneous coprocessors for GEMM and GEMV are desired. The performance of compute-bound GEMM can be enhanced by increasing computing resources, while memory-bound GEMV requires addressing bandwidth.

Insight 2: Activation-aware weights pruning is promising. Many prior work focuses on long-context KV cache optimization [42]. For short-context edge MLLMs, the weight matrices of FFN are key constraints, while the sparsity in most activation channels offers great potential for optimization.

Insight 3: Scheduling for output token lengths is needed. Since the workload proportion varies with the output token length, dynamic scheduling is also strongly desired to maintain the balanced workload latency of heterogeneous cores.

III. EDGE MM ARCHITECTURE DESIGN

A. Architecture Overview

To accommodate the mentioned characteristics of edge MLLMs, we present a multi-core CPU architecture, i.e. EdgeMM, with heterogeneous AI extensions, as shown in Fig. 4. EdgeMM adopts a hierarchical multi-core architecture based on the open-source Snitch_cluster [43]. Two types of accelerator architecture are incorporated as coprocessors for compute-centric (CC) cores and memory-centric (MC) cores. Specifically, CC-cores utilize compute-dense systolic array (SA)-based coprocessors for GEMM. While MC-cores utilize memory-abundant digital CIM, which integrates the compute units within SRAM banks, as coprocessors for GEMV.

For CC-cores, cores in a cluster share the instruction memory and data memory. In contrast, the data memory and

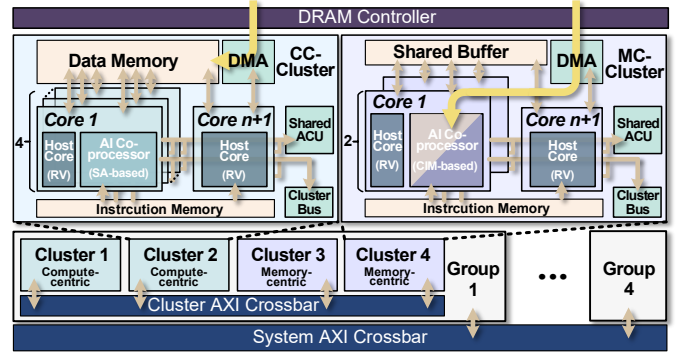


Fig. 4. Architecture overview of EdgeMM design.

compute array in MC-cores are integrated as a CIM macro in each core, and a small shared buffer is equipped for inter-core data transfer. Every cluster has a distributed DMA module, all of which are linked to the DRAM controller for data transfer to on-chip data memory. Auxiliary compute units (ACU), e.g. 32-bit multiplier and divider, are also shared among cores in a cluster for uncommon calculations.

The entire chip contains 4 groups, each consisting of 2 CC-clusters and 2 MC-clusters. Each CC-cluster and MC-cluster contains 4 CC-cores and 2 MC-cores respectively. These cores integrate an area-efficient RISC-V host core [43] for control and an extended coprocessor for AI computing. Each cluster also has a dedicated host core to control the DMA module. The groups and clusters are connected by the hierarchical AXI crossbars. The hardware architecture can also be scaled by changing architecture parameters.

B. CPU Extension for AI Acceleration

Fig. 5 and Fig. 6 illustrate the design details of CC and MC cores. Both core types consist of a host core and a coprocessor. The extended instructions are decoded by host core and dispatched to coprocessor via direct-linked interface.

Compute-Centric core: As shown in Fig. 5, the bulk of compute-centric GEMM coprocessor is a systolic array, which contains $R \times C$ multiply-accumulate processing elements (PE). For GEMM operations, the weights stored in the PE array remain stationary, while the input activations are sent into the array in a systolic manner. Four $R \times C$ matrix registers are equipped to store operands. The vector units are employed to execute vector instructions for element-wise computations. Vector instructions share the matrix registers and have an element width of C , enabling parallel operation on a row of a matrix register by one instruction. The coprocessor has an independent load/store unit for matrix registers, avoiding the data bottleneck caused by the narrow port of the host core. For multiplication between a $R \times C$ and an $M \times R$ matrix, the cycles of loading and computing equals:

$$L_{SA} = R + (R - 1) + (C + M - 1) - 1 = 2R + C + M - 3 \quad (2)$$

Systolic dataflow enables data transfer solely between neighboring PEs, reducing wiring overhead. However, there is only one column of activation (circled in red in Fig. 5) in GEMV, causing PEs idleness and inefficiency.

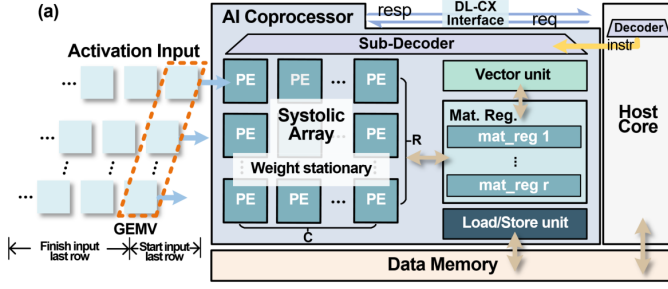


Fig. 5. Compute-centric core with Systolic Array.

Memory-Centric core: As shown in Fig. 6, the main feature of an MC-core is utilizing digital CIM for the coprocessor, which integrates compute cells within SRAM macro to avoid reg-to-SRAM load/store. A CIM macro consists of a controller, wordline decoder, write-and-read circuits, and C columns. Each column contains R subarrays, an adder tree, and a shift-and-accumulator. Each subarray consists of $M \times N$ 6T SRAM bit-cells. Here N equals to the bit-width of a weight data. During GEMV, the W -bit activation data is broadcasted into the columns in bit-serial. One of the weights in each subarray is read out and multiplied with 1-bit activation every cycle. Adder tree sums the R product and sends the result to shift-and-accumulator, in which the sum is shifted and accumulated to finish the entire W -bit multiplication. The broadcast dataflow fully utilizes the compute cells for GEMV, which is completed in $W+1$ cycles. The cycle number of a M rows GEMM is:

$$L_{CIM} = M * W + 1 \quad (3)$$

Compared to SA, CIM requires fewer cycles for GEMV, i.e. $M = 1$ in (2)(3). But for GEMM, CIM is less efficient due to the bit-width factor W . Moreover, MC-clusters have significantly larger data memory than CC-clusters, which allows larger matrix block to be transferred at once. We assess the effective bandwidth (data amount/cycles) for transferring matrices with various sizes. Fig. 6 (b) shows that the effective bandwidth drops notably for small matrices, but nears the ideal bandwidth as matrix size increases. This indicates the ample on-chip memory in MC-cluster can alleviate the bandwidth pressure and enhance DMA and DRAM efficiency.

C. ISA Extension and Programming Model

We adopt RISC-V as our main ISA and extend instructions for AI acceleration as shown in Fig 7. The first row illustrates the matrix instruction (i.e. `mm_mul` for GEMM) supported by CC-core. The matrix instructions differ from scalar instructions by employing matrix registers for both the source and destination operands. The CIM-based MC-core supports matrix-vector instructions, in which the `vs1` and `vd` are vector registers that store source and destination operands, and the `rs1` store the base address of matrix operand. The vector units in all cores support a subset of RISC-V vector instructions for activation functions and data precision conversion. Configure instructions can modify the control and status registers (CSR), storing the runtime parameters such as vector/matrix sizes.

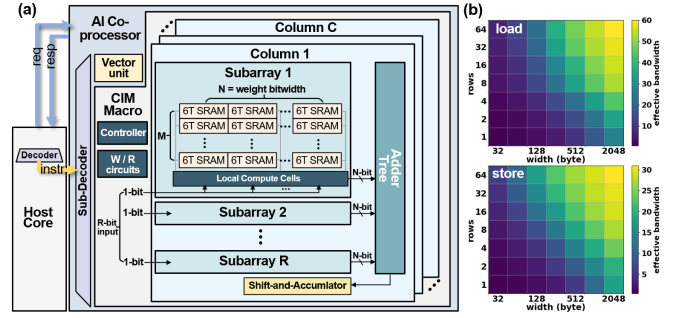


Fig. 6. Mem-centric core with CIM macro design.

M-M	31	func	27,26	25	23	21,20	18,17	15,14	12,11	10,9	7,6	major opcode	0
		uop			ms2	ms1	md	func3	size	uimm			
M-V	31	func	27,26	25,24		20,19		15,14	12,11		7,6	major opcode	0
		uop			vs1	rs1		func3	vd				
V-V	31	func	27,26	25,24		20,19		15,14	12,11	10,9	7,6	major opcode	0
		uop			vs2	vs1		func3	vd				
Config	31	func	27,26	25,24		20,19		15,14	12,11	10,9	7,6	major opcode	0
		uop			rs1	CSR		func3	size	null			

Fig. 7. Instruction format of the extended instructions.

The programming model of our EdgeMM is similar to `snitch_cluster` [43]. The computing tasks, e.g. GEMM/GEMV, are allocated across cores with tensor partitioning. Specifically, each core and cluster contains read-only CSRs that store its index and type, allowing cores to calculate address offsets to access the allocated tensor shards. The core synchronization function is also supported. For ISA extension, the extended instructions can be utilized by customized kernel functions, enabling the use of the RISC-V toolchain without the internal modification of the compiler.

IV. BANDWIDTH OPTIMIZATIONS

To effectively utilize the limited bandwidth, we optimize memory access reduction by activation-aware weight pruning and perform token-length-driven bandwidth management.

A. Activation-aware Weight Pruning for GEMV

As Sec. II-B mentioned, the FFN weight matrices dominate DRAM access, and the activation vectors exhibit significant sparsity and outliers across channels. This motivates us to perform channel-wise activation-aware pruning. As shown in Fig. 8, GEMV can be performed by selecting the Top- k outliers and pruning the rest channels with corresponding rows in weight matrices. Recent works [44], [45] adopt the similar concept, but they all regard k as a fixed empirical input. In fact, k can vary along different decoder layers for better pruning. For example, early decoder layers should retain most weights to avoid errors accumulating and the latter layers have more distinct outliers, allowing for more efficient pruning.

As shown in Alg. 1, a layer-wise dynamic Top- k pruning scheme is presented. The update of k is based on straightforward yet effective principles: a channel that is t times smaller than the max channel can be considered negligible, where t is a fixed parameter set to 16 in our design. Moreover, k should decrease progressively with layer depth. If the number of retained channels n is less than k , it indicates that the current k is too large. Thus, k is set to n .

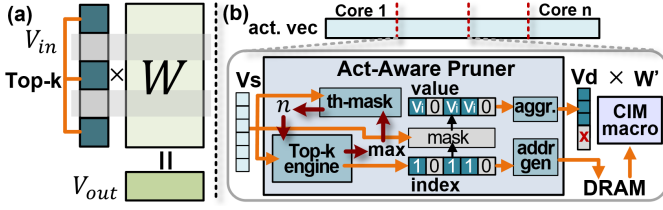


Fig. 8. Activation-aware weight pruning and hardware pruner.

Algorithm 1: Layer-wise Dynamic Top-k Pruning

```

input : Threshold  $t$ , Vector dim  $d$ 
for layers in model do
  if layer_index == 1 then
     $k = d$ ; // no pruning
     $V'_x = \text{top-k}(V_x, k)$ ; // select channels
     $W' = \text{pruning}(W, \text{index}(V'_x))$ ; // pruning
     $\text{GEMV}(W', V'_x)$ 
     $n = \text{count}(V_x[i] \geq \frac{\max(V_x[i])}{t})$ 
    if  $n < k$  then
       $k = n$ ; // update  $k$ 
    /* remaining computation */
  generate an output token

```

The hardware pruner in the MC-core facilitates the execution of GEMV with Alg. 1, as shown in Fig 8 (b). In practice, the activation vector is allocated to cores by channels. Each core focuses on its assigned local channels, avoiding complex global Top-k selections. The core first loads a slice of the activation vector into the vector register vs , then calls the hardware Act-Aware pruner for processing via specific instruction. In pruner, the Top-k engine finds the $\max$ value in vs and marks the selected elements with 1 in the index register. The th-mask receives the $\max$ value and vs , generating n in Alg. 1 for k update. The index is fed into the address generator, which calculates the address of the non-pruned weight rows for DRAM read requests. Following that, the vs is masked by the index and aggregated, and then output to vd register. Then the CIM macro computes the GEMV between vd and stored weights. The above process reduces DRAM access without wasting on-chip computation or memory resources.

B. Token-Length-Driven Bandwidth Management

The heterogeneity of EdgeMM complicates workload distribution. For MLLMs, it is optimal to run modality encoder and LLM-prefill on CC-clusters, with LLM-decoding on MC-clusters. In real-time applications (e.g. AD, robot, AR/VR), continuous streaming input enables pipelined execution. But the latency of LLM-decoding increases with the output token length, which can result in an unbalanced pipeline.

To maintain a balanced pipeline for various token lengths l , a throttling-based dynamic bandwidth allocation is adopted. For a model with default equal bandwidth sharing among clusters, an expected token length l_e can balance the latency of CC and MC clusters. As shown in Fig. 9(a), if l exceeds

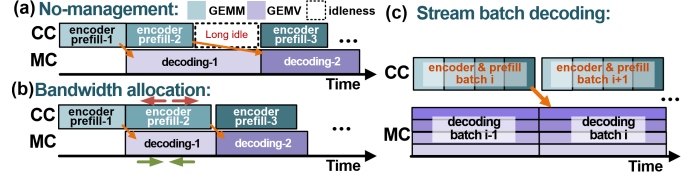


Fig. 9. Pipeline of bandwidth and workload management.

(a)	CC-cluster	MC-cluster
Cores	4	2
Array(R,C)	(16,16)	(32,16)
OPs/s	2T	0.25T
Data Mem	32kB	128kB
DMA width	512bit	512bit
Crossbar width	1024bit	512bit
Area	0.619mm ²	0.652mm ²
Clusters in a group	2	2

(b)

Fig. 10. Design configurations and the 22nm layout of EdgeMM.

l_e , CC-cluster will enter waiting idle. In such scenarios in Fig. 9(b), limiting CC-cluster bandwidth and reallocating it to MC-clusters can accelerate LLM-decoding, reducing overall latency. To implement bandwidth allocation, each cluster is assigned a memory access budget B . Within an interval T , the performance monitoring counter (PMC) in DMA accumulates the memory access usage d of each cluster. If $d > B$, subsequent DMA requests from this cluster are blocked until T elapses and PMCs are reset. We adjust the cluster budget B_c and B_m to enable dynamic bandwidth allocation.

Note the bandwidth allocation can only balance the pipeline with l in a certain range. For larger l , stream-based batch processing can be employed. As shown in Fig 9 (c), CC-clusters can encode and prefill a batch of streaming input without waiting for LLM-decoding. The MC cluster performs decoding concurrently on this batch. Larger batch multiplies the latency of CC-clusters, while decoding latency of MC-clusters increases slightly for the weight reuse. Hence the overall throughput can be boosted with minimal latency loss.

V. EVALUATIONS

A. Implementation and Evaluation Setup

We leverage open-source Snitch_cluster [43] as the foundation platform for our EdgeMM development. The CC-cluster and MC-cluster are implemented using RTL flow and in-house 22nm digital CIM macro IP. We utilize Cadence Genus and Cadence Innovus for synthesis and physical implementation at 1GHz using TSMC 22nm technology. Fig. 10 demonstrates the layout view and design configurations of EdgeMM. The chip power is 112mW according to post-P&R report. The SA-based coprocessor occupies 62% area of a CC-core, and the CIM macro occupies 81% area of an MC-core. The performance is evaluated by RTL simulation and the in-house simulator with a dedicated mapping explorer.

B. Benefits of Heterogeneous Extensions

To evaluate the benefits of our heterogeneous AI-extensions, we compare our heterogeneous EdgeMM with homogeneous cluster designs: homo-CC and homo-MC, which consist solely of CC-clusters and MC-clusters. The original snitch_cluster

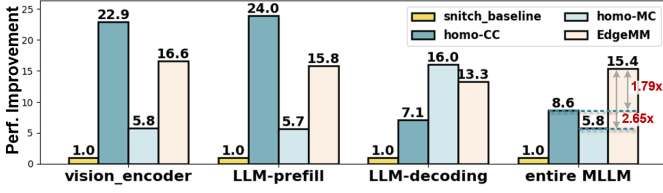


Fig. 11. Perf. improvements of homo. and hetero. designs

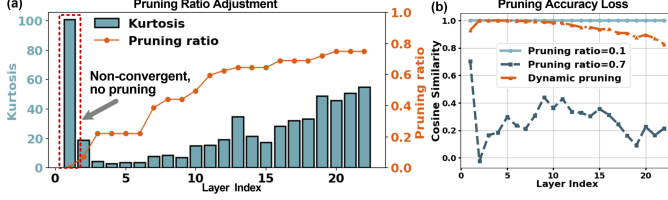


Fig. 12. Evaluation of activation-aware dynamic Top-k weight pruning.

[43] including SIMD cores is set as the baseline. The SPHINX-Tiny [25] MLLM and its inner phases with averaged input and output token length are used as the workload.

As shown in Fig. 11, all extended designs have significant performance boosts compared to the baseline since matrix extensions and coprocessors enhance computing power and reduce redundant load/store. Homo-CC and homo-MC perform the peak speedup in their adapted phases: a CC-cluster shows $4.3\times$ better GEMM performance than an MC-cluster, whereas an MC-cluster is $2.42\times$ faster in GEMV due to more efficient memory access. However, homo-CC and homo-MC encounter bottlenecks in non-adapted phases due to hardware mismatch. For the entire MLLM, the heterogeneous EdgeMM outperforms them by $1.79\times$ and $2.65\times$ respectively.

C. Benefits of Bandwidth Optimizations

Fig. 12 (a) presents the evaluation results for activation-aware dynamic TOP-k weight pruning during a token generation, where *Kurtosis* is a mathematical metric to represent channel-wise data distribution. Higher *Kurtosis* indicates more distinct outliers, i.e. more channels can be pruned. Results show that with *Kurtosis* increasing with layer depth, with pruning ratio of a randomly chosen core increases dynamically, proving the activation-awareness of dynamic TOP-k mechanism. Note that although the first layer exhibits a high *Kurtosis*, its data distribution remains unstable. Our experiment reveals that pruning this layer leads to significant accuracy degradation. Thus Alg. 1 skips the first layer.

To validate the accuracy of dynamic TOP-k pruning, we assess the cosine similarity between pruned and unpruned output vectors in FFN as depicted in Fig.12 (b). It is observed that dynamic pruning achieves comparable accuracy as a mild fixed pruning ratio of 0.1 while fixing pruning ratio at 0.7 causes irreversible accuracy loss in the shallow layers. Overall, our activation-aware pruning reduces the LLM-decoding latency of SPHINX-Tiny by 42% on average, with minimal score reduction in VQA [35].

Our dynamic bandwidth allocation can harness the heterogeneous cores under varying output token lengths l . As described in Fig. 13, when $l < l_e = 36$, bandwidth is not the critical

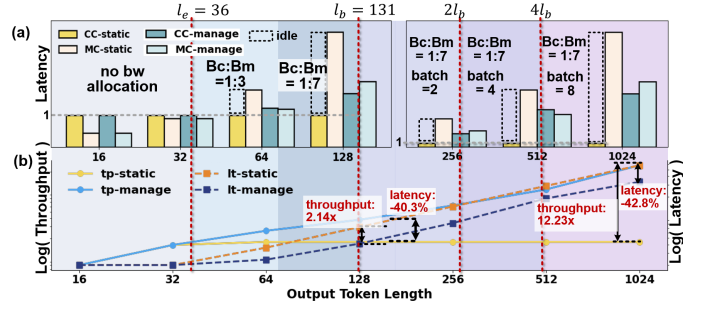


Fig. 13. (a) Latency and (b) throughput gains by bandwidth management.

bottleneck as the latency of CC-cores is large. As l rises, the extended duration of LLM-decoding slows down the pipeline. Thus the bandwidth budget of CC-cores B_c is reduced progressively, reallocating bandwidth to MC-cores. The $B_c:B_m$ ratio ranges to 1:3 or even 1:7, accelerating MC-cores with tolerable latency increment of CC-cores. This optimization reduces latency by 40.3% and improves throughput $2.14\times$ at $l=128$. When l increases to $l_b = 131$, bandwidth allocation nears its limit, where batch decoding is introduced to balance the pipeline. While batch decoding increases latency by 42% at $l = 1024$, it boosts throughput by $13.98\times$, compensating for the latency overhead.

We further compare EdgeMM with existing mobile GPU, as shown in Table II. Although SM cores in GPU provide massive parallelism, they often remain underutilized and not as effective as SA and CIM extensions in EdgeMM. Compared to laptop RTX 3060 GPU, EdgeMM achieves $2.15\times$ performance for better resource utilization driven by heterogeneity. With our activation-aware weight pruning, the performance gain can be enhanced to $2.84\times$, reaching 138 tokens/s throughput. Overall, EdgeMM achieves a 0.28 token/J energy efficiency.

TABLE II
COMPARISON OF EDGEMM AND MOBILE GPU

	Compute Power	Bandwidth	MLLM perf.
RTX 3060 Laptop	13TFLOP/s (FP32)		1×
EdgeMM	18TFLOP/s (BF16)	GDDR6	2.15×
EdgeMM+Weight pruning	18TFLOP/s (BF16)	336GB/s	2.84×

VI. CONCLUSION

We present EdgeMM, a multi-core CPU with heterogeneous AI-extension for edge MLLM. To accommodate phases with diverse attributes in MLLM, coprocessors based on systolic arrays and digital CIM macros are integrated for GEMM and GEMV. To leverage limited bandwidth, activation-aware dynamic weight pruning is proposed with hardware support. Token-length-driven bandwidth allocation is employed to fully utilize heterogeneous cores. Overall, EdgeMM achieves $2.84\times$ performance improvement compared to RTX 3060 GPU.

ACKNOWLEDGMENT

This work is supported in part by NSFC Grant No. 92464202 and CIE-Smartchip research fund.

REFERENCES

- [1] C. Cui *et al.*, “A survey on multimodal large language models for autonomous driving,” in *IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 958–979, 2024.
- [2] Y. Liu, W. Chen, Y. Bai, J. Luo, X. Song, K. Jiang, Z. Li, G. Zhao, J. Lin, G. Li, *et al.*, “Aligning cyber space with physical world: A comprehensive survey on embodied ai,” *arXiv preprint arXiv:2407.06886*, 2024.
- [3] X. Chu, L. Qiao, X. Lin, S. Xu, Y. Yang, Y. Hu, F. Wei, X. Zhang, B. Zhang, X. Wei, *et al.*, “Mobilevlm: A fast, reproducible and strong vision language assistant for mobile devices,” *arXiv preprint arXiv:2312.16886*, 2023.
- [4] Y.-C. Chen, W.-H. Li, C. Sun, Y.-C. F. Wang, and C.-S. Chen, “Sam4mllm: Enhance multi-modal large language model for referring expression segmentation,” *arXiv preprint arXiv:2409.10542*, 2024.
- [5] H. Shao *et al.*, “Lmdrive: Closed-loop end-to-end driving with large language models,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15120–15130, 2024.
- [6] B. Keller *et al.*, “A 17–95.6 tops/w deep learning inference accelerator with per-vector scaled 4-bit quantization for transformers in 5nm,” in *2022 IEEE Symposium on VLSI Technology and Circuits (VLSI)*, 2022.
- [7] T. Tambe *et al.*, “22.9 a 12nm 18.1tflops/w sparse transformer processor with entropy-based early exit, mixed-precision predication and fine-grained power management,” in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2023.
- [8] Z. XUANLEI, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, “Hetegen: Efficient heterogeneous parallel inference for large language models on resource-constrained devices,” in *Machine Learning and Systems* (P. Gibbons, G. Pekhimenko, and C. D. Sa, eds.), vol. 6, pp. 162–172, 2024.
- [9] C. Silvano, D. Ielmini, F. Ferrandi, L. Fiorin, S. Curzel, L. Benini, F. Conti, A. Garofalo, C. Zambelli, E. Calore, *et al.*, “A survey on deep learning hardware accelerators for heterogeneous hpc platforms,” *arXiv preprint arXiv:2306.15552*, 2023.
- [10] Z. Jiang, N. Fisher, N. Guan, and Z. Dong, “Blueface: Integrating an accelerator into the core’s pipeline through algorithm-interface co-design for real-time socs,” in *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pp. 1–6, 2023.
- [11] Online in <https://www.intel.com/content/www/us/en/products/docs/accelerator-engines/advanced-matrix-extensions/overview.html>.
- [12] N. Nassif *et al.*, “Sapphire rapids: The next-generation intel xeon scalable processor,” in *IEEE International Solid-State Circuits Conference (ISSCC)*, pp. 44–46, 2022.
- [13] Y. Yuan *et al.*, “Intel accelerators ecosystem: An soc-oriented perspective,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 848–862, 2024.
- [14] Online in <https://developer.arm.com/documentation/ddi0616/latest/>.
- [15] Online in <https://github.com/T-head-Semi/riscv-matrix-extension-spec>.
- [16] Q. Sun, Y. Cui, X. Zhang, F. Zhang, Q. Yu, Y. Wang, Y. Rao, J. Liu, T. Huang, and X. Wang, “Generative multimodal models are in-context learners,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14398–14409, 2024.
- [17] Q. Sun, Y. Fang, L. Wu, X. Wang, and Y. Cao, “Eva-clip: Improved training techniques for clip at scale,” *arXiv preprint arXiv:2303.15389*, 2023.
- [18] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” *Advances in neural information processing systems*, vol. 36, 2024.
- [19] A. Radford *et al.*, “Learning transferable visual models from natural language supervision,” in *International Conference on Machine Learning (ICML)*, pp. 8748–8763, 2021.
- [20] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez, *et al.*, “Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, march 2023,” *URL https://lmsys.org/blog/2023-03-30-vicuna*, vol. 3, no. 5, 2023.
- [21] X. Chu, L. Qiao, X. Zhang, S. Xu, F. Wei, Y. Yang, X. Sun, Y. Hu, X. Lin, B. Zhang, *et al.*, “Mobilevlm v2: Faster and stronger baseline for vision language model,” *arXiv preprint arXiv:2402.03766*, 2024.
- [22] Z. Yuan, Z. Li, and L. Sun, “Tinygpt-v: Efficient multimodal large language model via small backbones,” *arXiv preprint arXiv:2312.16862*, 2023.
- [23] J. Li, D. Li, S. Savarese, and S. Hoi, “Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models,” in *International Conference on Machine Learning (ICML)*, pp. 19730–19742, 2023.
- [24] M. Javaheripi, S. Bubeck, M. Abdin, J. Aneja, S. Bubeck, C. C. T. Mendes, W. Chen, A. Del Giorno, R. Eldan, S. Gopi, *et al.*, “Phi-2: The surprising power of small language models,” *Microsoft Research Blog*, 2023.
- [25] P. Gao, R. Zhang, C. Liu, L. Qiu, S. Huang, W. Lin, S. Zhao, S. Geng, Z. Lin, P. Jin, *et al.*, “Sphinx-x: Scaling data and parameters for a family of multi-modal large language models,” *arXiv preprint arXiv:2402.05935*, 2024.
- [26] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, *et al.*, “Dinov2: Learning robust visual features without supervision,” *arXiv preprint arXiv:2304.07193*, 2023.
- [27] P. Zhang, G. Zeng, T. Wang, and W. Lu, “Tinyllama: An open-source small language model,” *arXiv preprint arXiv:2401.02385*, 2024.
- [28] H. Lu, W. Liu, B. Zhang, B. Wang, K. Dong, B. Liu, J. Sun, T. Ren, Z. Li, Y. Sun, *et al.*, “Deepseek-vl: towards real-world vision-language understanding,” *arXiv preprint arXiv:2403.05525*, 2024.
- [29] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 11975–11986, 2023.
- [30] Online in *Karmavlm: A family of high efficiency and powerful visual language model*. <https://github.com/thomas-yanxin/KarmaVLM>, 2024.
- [31] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, *et al.*, “Qwen technical report,” *arXiv preprint arXiv:2309.16609*, 2023.
- [32] G. Team, R. Anil, S. Borgeaud, Y. Wu, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, *et al.*, “Gemini: a family of highly capable multimodal models,” *arXiv preprint arXiv:2312.11805*, 2023.
- [33] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat, *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [34] H. Liu, C. Li, Y. Li, and Y. J. Lee, “Improved baselines with visual instruction tuning,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 26296–26306, 2024.
- [35] Y. Goyal, T. Khot, A. Agrawal, D. Summers-Stay, D. Batra, and D. Parikh, “Making the v in vqa matter: Elevating the role of image understanding in visual question answering,” 2019.
- [36] P. Lu, S. Mishra, T. Xia, L. Qiu, K.-W. Chang, S.-C. Zhu, O. Tafjord, P. Clark, and A. Kalyan, “Learn to explain: Multimodal reasoning via thought chains for science question answering,” *ArXiv*, vol. abs/2209.09513, 2022.
- [37] B. Li, R. Wang, G. Wang, Y. Ge, Y. Ge, and Y. Shan, “Seed-bench: Benchmarking multimodal llms with generative comprehension,” *ArXiv*, vol. abs/2307.16125, 2023.
- [38] Y. Liu, H. Duan, Y. Zhang, B. Li, S. Zhang, W. Zhao, Y. Yuan, J. Wang, C. He, Z. Liu, K. Chen, and D. Lin, “Mmbench: Is your multi-modal model an all-around player?,” in *European Conference on Computer Vision*, 2023.
- [39] S. Kim, C. Hooper, T. Wattanawong, M. Kang, R. Yan, H. Genc, G. Dinh, Q. Huang, K. Keutzer, M. W. Mahoney, *et al.*, “Full stack optimization of transformer inference: a survey,” *arXiv preprint arXiv:2302.14017*, 2023.
- [40] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [41] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, *et al.*, “Mistral 7b,” *arXiv preprint arXiv:2310.06825*, 2023.
- [42] Z. A. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. W. Barrett, Z. Wang, and B. Chen, “H2o: Heavy-hitter oracle for efficient generative inference of large language models,” *ArXiv*, vol. abs/2306.14048, 2023.
- [43] F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini, “Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads,” *IEEE Transactions on Computers*, 2020.
- [44] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, “A simple and effective pruning approach for large language models,” *arXiv preprint arXiv:2306.11695*, 2023.
- [45] J.-Y. Lee, D. Lee, G. Zhang, M. Tiwari, and A. Mirhoseini, “Cats: Contextually-aware thresholding for sparsity in large language models,” *arXiv preprint arXiv:2404.08763*, 2024.